Rick & Morty

Android Demo App — Technical Specification

MVI Architecture | Clean Architecture | Jetpack Compose | Retrofit · Room

Feature list · Screens · Navigation · API reference with real request & response

Data source: https://rickandmortyapi.com/api

Version 1.0 · August 2026

Rick & Morty App — Technical Spec

A single reference for building the demo app: what to build (features), what the user sees (screens), how they move (navigation), and how the app talks to the API (endpoints with real sample request & response bodies).

Contents

1. API Overview & Conventions

The app consumes the public **Rick and Morty REST API**. No authentication or API key is required. All requests are `GET` only — the API is read-only.

Property	Value
Base URL	`https://rickandmortyapi.com/api`
Auth	None (public, read-only)
Format	JSON
Resources	Character (826), Location (126), Episode (51)
Page size	20 items per page (fixed)
Pagination param	`?page={n}`

Pagination — the `info` object

Every list endpoint wraps results in an `info` object plus a `results` array. Use `info.next` to drive infinite scroll / Paging 3 — when `next` is `null`, there are no more pages.

```
{
  "info": {
    "count": 826,       // total items across all pages
    "pages": 42,        // total number of pages
    "next": "https://rickandmortyapi.com/api/character?page=2",
    "prev": null        // null on the first page
  },
  "results": [ /* up to 20 objects */ ]
}
```

> **Filtering:** filters are query params on the list endpoint and can be combined, e.g. `/character/?status=alive&gender=female&name=beth`. A filter that matches nothing returns HTTP `404` with an error body (see §8).

2. Feature List

Character features

Feature	Endpoint used
Character list with infinite scroll / pagination	`GET /character?page={n}`
Character detail (image, status, species, gender, origin, location)	`GET /character/{id}`
Search characters by name	`GET /character/?name={q}`
Filter by status / species / gender	`GET /character/?status=alive&gender=female`
Episodes a character appears in	from the `episode[]` array on the character

Episode features

Feature	Endpoint used
Episode list	`GET /episode?page={n}`
Episode detail (name, air date, code)	`GET /episode/{id}`
Search episode by name or code	`GET /episode/?name={q}` · `?episode=S01E01`
Characters featured in an episode (grid)	`GET /character/{1,2,3}` from `characters[]`

Location features

Feature	Endpoint used
Location list	`GET /location?page={n}`
Location detail (type, dimension)	`GET /location/{id}`
Search / filter by name, type, dimension	`GET /location/?type=Planet`
Residents of a location (grid)	`GET /character/{1,2,3}` from `residents[]`

Cross-cutting features (architecture showcase)

Feature	Layer / mechanism
Favorites — bookmark characters	Local **Room** database
Offline caching — list survives no network	Room + **Paging 3 RemoteMediator**
Pull-to-refresh on any list	Presentation + repository refresh
Batch fetch of related items	`/character/1,2,3` multi-ID endpoint
Loading / empty / error states	MVI `State` per screen

> **Suggested MVP:** Character list + search/filter + detail + favorites (Room). That alone exercises networking, local DB, DTO→domain mapping, use cases and MVI state end-to-end. Episodes and Locations then reuse the exact same patterns.

3. Screen List

#	Screen	Purpose	Primary data
S1	Character List	Paginated grid/list of characters; entry point tab	`GET /character`
S2	Character Search & Filter	Search bar + filter chips (status, gender, species)	`GET /character/?name=&status=…`
S3	Character Detail	Full profile, origin, last location, episode list	`GET /character/{id}`
S4	Episode List	Paginated list grouped by season	`GET /episode`
S5	Episode Detail	Air date, code, grid of featured characters	`GET /episode/{id}` + `/character/{ids}`
S6	Location List	Paginated list of locations	`GET /location`
S7	Location Detail	Type, dimension, grid of residents	`GET /location/{id}` + `/character/{ids}`
S8	Favorites	Locally saved characters (offline)	Room DB

Bottom navigation tabs: **Characters** · **Episodes** · **Locations** · **Favorites**. Detail screens are pushed on top of the active tab's back stack.

4. Navigation Map

Single-Activity app with Jetpack Compose Navigation. Four top-level destinations in a bottom bar; detail screens receive an `id` argument.

```
App (Scaffold + bottom bar)
│
├─ [Tab] Characters ──► CharacterList (S1)
│                          │  search/filter (S2, same screen)
│                          └─► CharacterDetail (S3)  { characterId }
│                                 └─► EpisodeDetail (S5)   // tap an episode
│
├─ [Tab] Episodes ──► EpisodeList (S4)
│                          └─► EpisodeDetail (S5)  { episodeId }
│                                 └─► CharacterDetail (S3)  // tap a character
│
├─ [Tab] Locations ──► LocationList (S6)
│                          └─► LocationDetail (S7)  { locationId }
│                                 └─► CharacterDetail (S3)  // tap a resident
│
└─ [Tab] Favorites ──► FavoritesList (S8)
                          └─► CharacterDetail (S3)  { characterId }
```

Route definitions (Compose)

```
characters                    // S1 / S2
character/{characterId}       // S3   arg: Int
episodes                      // S4
episode/{episodeId}           // S5   arg: Int
locations                     // S6
location/{locationId}         // S7   arg: Int
favorites                     // S8
```

CharacterDetail is reachable from four places (list, episode, location, favorites) — define it once and navigate to it with the character id from each source.

5. API Reference — Characters

Fields: `id`, `name`, `status`, `species`, `type`, `gender`, `origin`, `location`, `image`, `episode[]`, `url`, `created`.

GET `/character?page=1` list · paginated

Screen S1 — the character list. Returns `info` + up to 20 characters. Drive infinite scroll from `info.next`.

Request

```
GET https://rickandmortyapi.com/api/character?page=1
Accept: application/json
```

Response — 200 OK (info + first result shown; array holds 20)

```
{
  "info": {
    "count": 826,
    "pages": 42,
    "next": "https://rickandmortyapi.com/api/character?page=2",
    "prev": null
  },
  "results": [
    {
      "id": 1,
      "name": "Rick Sanchez",
      "status": "Alive",
      "species": "Human",
      "type": "",
      "gender": "Male",
      "origin": {
        "name": "Earth (C-137)",
        "url": "https://rickandmortyapi.com/api/location/1"
      },
      "location": {
        "name": "Citadel of Ricks",
        "url": "https://rickandmortyapi.com/api/location/3"
      },
      "image": "https://rickandmortyapi.com/api/character/avatar/1.jpeg",
      "episode": [
        "https://rickandmortyapi.com/api/episode/1",
        "https://rickandmortyapi.com/api/episode/2"
        /* … 51 episode URLs total … */
      ],
      "url": "https://rickandmortyapi.com/api/character/1",
      "created": "2017-11-04T18:48:46.250Z"
    }
    /* … 19 more character objects … */
  ]
}
```

GET `/character/{id}` single

Screen S3 — character detail. Returns one character object.

Request

```
GET https://rickandmortyapi.com/api/character/1
```

Response — 200 OK


```
{
  "id": 1,
  "name": "Rick Sanchez",
  "status": "Alive",
  "species": "Human",
  "type": "",
  "gender": "Male",
  "origin": {
    "name": "Earth (C-137)",
    "url": "https://rickandmortyapi.com/api/location/1"
  },
  "location": {
    "name": "Citadel of Ricks",
    "url": "https://rickandmortyapi.com/api/location/3"
  },
  "image": "https://rickandmortyapi.com/api/character/avatar/1.jpeg",
  "episode": [
    "https://rickandmortyapi.com/api/episode/1",
    "https://rickandmortyapi.com/api/episode/2",
    "https://rickandmortyapi.com/api/episode/3"
    /* … up to episode/51 … */
  ],
  "url": "https://rickandmortyapi.com/api/character/1",
  "created": "2017-11-04T18:48:46.250Z"
}
```


GET `/character/1,2,3` multiple ids

Batch fetch — used on S5/S7 to load the characters listed in an episode's `characters[]` or a location's `residents[]`. Returns an **array** (not wrapped in `info`). A single id returns an object, so treat 1-id lists carefully.

Request

```
GET https://rickandmortyapi.com/api/character/1,2,3
```

Response — 200 OK


```
[
  { "id": 1, "name": "Rick Sanchez", "status": "Alive", "...": "..." },
  { "id": 2, "name": "Morty Smith",  "status": "Alive", "...": "..." },
  { "id": 3, "name": "Summer Smith", "status": "Alive", "...": "..." }
]
```

GET `/character/?status=alive&gender=female` filter

Screen S2 — search & filter. Combine any of: `name`, `status` (alive/dead/unknown), `species`, `type`, `gender` (female/male/genderless/unknown). Same paginated shape as the list.

Request

```
GET https://rickandmortyapi.com/api/character/?status=alive&gender=female&name=beth
```

Response — 200 OK (shape identical to the list endpoint)

```
{
  "info": { "count": 2, "pages": 1, "next": null, "prev": null },
  "results": [
    { "id": 4, "name": "Beth Smith", "status": "Alive",
      "species": "Human", "gender": "Female", "...": "..." }
  ]
}
```

6. API Reference — Episodes

Fields: `id`, `name`, `air_date`, `episode` (code, e.g. `S01E01`), `characters[]`, `url`, `created`.

GET `/episode?page=1` list · paginated

Screen S4 — episode list. 51 episodes across 3 pages.

Request

```
GET https://rickandmortyapi.com/api/episode?page=1
```

Response — 200 OK

```
{
  "info": {
    "count": 51,
    "pages": 3,
    "next": "https://rickandmortyapi.com/api/episode?page=2",
    "prev": null
  },
  "results": [
    {
      "id": 1,
      "name": "Pilot",
      "air_date": "December 2, 2013",
      "episode": "S01E01",
      "characters": [
        "https://rickandmortyapi.com/api/character/1",
        "https://rickandmortyapi.com/api/character/2"
        /* … 19 character URLs … */
      ],
      "url": "https://rickandmortyapi.com/api/episode/1",
      "created": "2017-11-10T12:56:33.798Z"
    }
    /* … 19 more episode objects … */
  ]
}
```

GET `/episode/{id}` single

Screen S5 — episode detail. Use `characters[]` to batch-load the featured cast via `/character/{ids}`.

Request

```
GET https://rickandmortyapi.com/api/episode/1
```

Response — 200 OK

```
{
  "id": 1,
  "name": "Pilot",
  "air_date": "December 2, 2013",
  "episode": "S01E01",
  "characters": [
    "https://rickandmortyapi.com/api/character/1",
    "https://rickandmortyapi.com/api/character/2",
    "https://rickandmortyapi.com/api/character/35",
    "https://rickandmortyapi.com/api/character/38"
    /* … 19 entries total … */
  ],
  "url": "https://rickandmortyapi.com/api/episode/1",
  "created": "2017-11-10T12:56:33.798Z"
}
```

GET `/episode/?name=&episode=` filter

Search by `name` or by episode `episode` code. Paginated shape.

Request

```
GET https://rickandmortyapi.com/api/episode/?episode=S01E01
GET https://rickandmortyapi.com/api/episode/?name=pilot
```

Response — 200 OK (same wrapper as the list)

```
{
  "info": { "count": 1, "pages": 1, "next": null, "prev": null },
  "results": [
    { "id": 1, "name": "Pilot", "air_date": "December 2, 2013",
      "episode": "S01E01", "...": "..." }
  ]
}
```

7. API Reference — Locations

Fields: `id`, `name`, `type`, `dimension`, `residents[]`, `url`, `created`.

GET `/location?page=1` list · paginated

Screen S6 — location list. 126 locations across 7 pages.

Request

```
GET https://rickandmortyapi.com/api/location?page=1
```

Response — 200 OK

```
{
  "info": {
    "count": 126,
    "pages": 7,
    "next": "https://rickandmortyapi.com/api/location?page=2",
    "prev": null
  },
  "results": [
    {
      "id": 1,
      "name": "Earth (C-137)",
      "type": "Planet",
      "dimension": "Dimension C-137",
      "residents": [
        "https://rickandmortyapi.com/api/character/38",
        "https://rickandmortyapi.com/api/character/45"
        /* … 27 resident URLs … */
      ],
      "url": "https://rickandmortyapi.com/api/location/1",
      "created": "2017-11-10T12:42:04.162Z"
    }
    /* … 19 more location objects … */
  ]
}
```

GET `/location/{id}` single

Screen S7 — location detail. Use `residents[]` to batch-load residents via `/character/{ids}`.

Request

```
GET https://rickandmortyapi.com/api/location/1
```

Response — 200 OK

```
{
  "id": 1,
  "name": "Earth (C-137)",
  "type": "Planet",
  "dimension": "Dimension C-137",
  "residents": [
    "https://rickandmortyapi.com/api/character/38",
    "https://rickandmortyapi.com/api/character/45",
    "https://rickandmortyapi.com/api/character/71",
    "https://rickandmortyapi.com/api/character/82",
    "https://rickandmortyapi.com/api/character/83"
    /* … 27 entries total … */
  ],
  "url": "https://rickandmortyapi.com/api/location/1",
  "created": "2017-11-10T12:42:04.162Z"
}
```

GET `/location/?type=Planet` filter

Filter by `name`, `type`, or `dimension`. Paginated shape.

Request

```
GET https://rickandmortyapi.com/api/location/?type=Planet&name=earth
```

Response — 200 OK (same wrapper as the list)

```
{
  "info": { "count": 4, "pages": 1, "next": null, "prev": null },
  "results": [
    { "id": 1, "name": "Earth (C-137)", "type": "Planet",
      "dimension": "Dimension C-137", "...": "..." }
  ]
}
```

8. Error Responses

The API returns a plain JSON body with a single `error` field. Map these to your MVI error state.

404 — single resource not found

```
GET /character/99999

{ "error": "Character not found" }
```

404 — filter / page matches nothing

```
GET /character/?name=zzzzzz

{ "error": "There is nothing here" }
```

> **Handling tip:** a `404` on a filter/search is a normal "empty result", not a crash — render an *empty state*, not an error dialog. Reserve the error state for network failures and 5xx.

Status	Meaning	UI response
200	OK	Render data
404	Not found / no matches	Empty state
500	Server error	Error state + retry
—	No connectivity (IOException)	Error state / cached data

9. Data Model Notes (DTO → Domain)

Each layer owns its own model. The DTO mirrors the JSON above; the domain model is what the UI and use cases consume. Mappers convert between them so a backend change never ripples into the UI.

JSON field	DTO type (Kotlin)	Domain model field	Note
`id`	Int	id: Int	Primary key
`name`	String	name: String	
`status`	String	status: CharacterStatus	map to enum: Alive/Dead/Unknown
`species`	String	species: String	
`gender`	String	gender: Gender	map to enum
`type`	String	type: String	often empty `""`
`origin`	OriginDto (name,url)	origin: LocationRef	nested object
`location`	LocationDto (name,url)	lastLocation: LocationRef	nested object
`image`	String	imageUrl: String	load with Coil
`episode`	List<String>	episodeIds: List<Int>	parse id from URL tail
`created`	String	createdAt: Instant	ISO-8601

Example Kotlin DTO

```
@Serializable
data class CharacterDto(
    val id: Int,
    val name: String,
    val status: String,
    val species: String,
    val type: String,
    val gender: String,
    val origin: LocationRefDto,
    val location: LocationRefDto,
    val image: String,
    val episode: List<String>,
    val url: String,
    val created: String
)

@Serializable
data class LocationRefDto(val name: String, val url: String)

@Serializable
data class PagedResponse<T>(val info: PageInfoDto, val results: List<T>)

@Serializable
data class PageInfoDto(
    val count: Int, val pages: Int,
    val next: String?, val prev: String?
)
```

> **ID from URL:** the API gives related items as full URLs (e.g. `.../episode/28`). Extract the trailing number — `url.substringAfterLast('/').toInt()` — to reuse the single/multiple endpoints.